# WORK IN PROGRESS

**Bachelor's Thesis**

by

*Jakob Meyerhöfer*

*(Mail: meyerhoe@rptu.de)*

February 27, 2026

University of Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze
Cass Alexandru (Gregor Cassian Alexandru), M.Sc.

## Declaration of Authorship

I hereby declare that the work I have submitted with the title „WORK IN PROGRESS“ was independently written by me. I have fully disclosed all sources and aids used and have identified as quotations all passages of the work — including tables and figures — taken verbatim or in substance from other works or the internet, indicating the respective sources.

Rodalben, on 27.2.2026

Jakob Meyerhöfer

# Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

## Summary

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

# Contents


**1. Introduction** **1**
- 1.1. Research Problem and Objectives . . . 2
- 1.2. Research Questions . . . 3
- 1.3. Thesis Structure . . . 3

**2. Background and Related Work** **7**
- 2.1. Challenges for Novice Programmers . . . 7
- 2.2. Error Messages and Programming Education . . . 7
- 2.3. Approaches to Improving Error Messages . . . 7
- 2.4. Case Study: Rust Error Messages? . . . 7

**3. Testify: Design and Goals** **9**
- 3.1. Design Principles for Pedagoically Useful Error Messages . . . 9
- 3.2. Using `Microsoft.FSharp.Quotations` and `Swensen.Unquote` for Expression Capture . . . 9
- 3.3. Assertify: Wrapper for Assertion . . . 9
- 3.4. Checkify: Wrapper for Property Based Tests . . . 9

**4. Evalutation and Results** **11**
- 4.1. Evaluation Methodology . . . 11
- 4.2. Results . . . 11
- 4.3. Threats to Validity . . . 11

**5. Discussion and Conclusion** **13**
- 5.1. Key findings . . . 13
- 5.2. Summary of Contributions . . . 13
- 5.3. Strengths and Limitations . . . 13
- 5.4. Future Work . . . 13

**List of Figures** **15**

**List of Listings** **17**

**List of Tables** **19**

**A. My Code** **21**

**B. My Ideas** **23**

# 1. Introduction

All programmers have to start at the beginning: What is programming? What is required to write and execute code? What are variables and methods and how are they used? From there, learners gradually encounter key concepts and algorithms, and later data structures such as maps and sets. However, moving from basic syntax to independently writing correct programs is a long process, which also includes implementing corresponding unit tests to validate solutions.

Understanding and interpreting error messages is a fundamental challenge for novice programmers. Error messages serve as an essential form of feedback, enabling programmers to diagnose issues in their code. In practice, standard diagnostics - especially those produced by compilers and testing frameworks - are often technical, verbose, and difficult for beginners to interpret.

This becomes particularly apparent in introductory programming courses where students receive exercise templates with predefined tests and are expected to implement missing functions or data structures. In such settings, learners are frequently unfamiliar with unit testing and are immediately confronted with stack traces and framework-specific output formats.

In `F#`, multiple testing frameworks and assertion libraries are available. A commonly used option is Microsoft.VisualStudio.TestTools.UnitTesting. For example, using `Assert.AreEqual` can yield the following output on failure:

```
Assert.AreEqual failed. Expected:<[1; 2; 4]>. Actual:<[1; 4; 2]>.

    at MapTests.TestsMapSortedList.Tests.testSetBeispiele() in /code/
    ↪  TestsMapSortedList.fs:line 53
    at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
    ↪  arguments, Signature sig, Boolean isConstructor)
    at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
    ↪  BindingFlags invokeAttr)
```

**Listing 1.1.** Example Ouput of `Assert.AreEqual`

For more advanced programmers, this output is usually sufficient: they can quickly identify the relevant mismatch and ignore the remaining details. See Section **??** (page **??**) for a discussion of common issues in diagnostic messages and strategies for extracting key information. In my experience as a tutor for a first-semester lecture, I frequently observe students struggling with such messages, unsure what went wrong and how to fix their code. Instead of guiding them toward a solution, poorly structured error messages can lead to frustration and trial-and-error debugging, which may hinder learning and delay progress on key programming concepts.

Didactically, complex concepts are typically introduced gradually, often by reducing incidental complexity at the beginning:

- Lightweight IDE's are often preferred to heavy, feature-rich IDE's
- Simplistic I/O is introduced before full input validation only valid inputs and error handling (e.g., malformed inputs, exceptions, and missing values).

The same Diagnostic verbosity can be staged for testing: Initially showing only the most relevant information and revealing additional technical details as students gain experience. This thesis explores how a similar scaffolding approach can be applied to testing feedback. By providing more minimalistic and structured output, the goal is to help students interpret failures more effectively and support a more efficient and engaging learning process.

## 1.1. Research Problem and Objectives

The primary problem addressed in this thesis is that, in introductory programming settings, the feedback produced by common testing frameworks is often not presented in a form that is immediately interpretable for novice programmers. In particular, test failures frequently mix the core mismatch information (what was expected versus what occurred) with framework-specific detail such as stack traces and internal method calls. As a consequence, beginners may struggle to identify the relevant part of the output and to translate it into concrete corrective actions in their own code.

From a technical perspective, this raises the question of how test feedback can be reformatted in a way that is both **(i)** more consistent and cognitively simpler for first-semester students and **(ii)** still compatible with established testing infrastructures. This thesis therefore focuses on the design and implementation of a lightweight assertion and reporting layer, *Assertify*, that produces a compact, structured summary of test failures. The objectives of this thesis are as follows:

- Analyse the structure and typical content of failing test output in beginner exercises, with a focus on which parts are essential for interpreting the failure.
- Design and implement a custom test/assertion library that enforces a consistent reporting scheme (e.g., a compact `expected = actual` summary) on top of existing frameworks.
- Reflect on design and implementation decisions by identifying current limitations of the library and outlining technically feasible extensions and future improvements.

To achieve these objectives, this thesis presents the implementation of *Assertify* as a wrapper around existing mechanisms for unit and property-based testing, and discusses how the reporting format can be further generalised, extended, and integrated into teaching workflows.

## 1.2. Research Questions

This thesis seeks to answer the following key research questions:

- **RQ1 (Problem characterisation):**

  Which structural characteristics of compiler and testing-framework outputs make failing tests difficult to interpret for novice programmers in introductory programming exercises?

- **RQ2 (Design requirements):**

  Which requirements can be derived for novice-oriented test feedback with respect to consistency, cognitive load, and the "actionability" of failure information?

- **RQ3 (Design proposal):**

  How can a uniform "tested expression / expected / actual" reporting scheme be defined such that it remains applicable across different kinds of tests (example-based unit tests and property-based tests)?

- **RQ4 (Technical feasibility):**

  How can such a reporting scheme be implemented in `F#` using Quotations and Unquote for expression capture and evaluation, while remaining compatible with existing tooling through wrappers around `TestTools.UnitTesting` and `FsCheck`?

- **RQ5 (Property-based integration):**

  How can property-based testing outcomes—including generated counterexamples and shrinking results—be represented within the same output format using `Checkify.CheckTest` and generators?

- **RQ6 (Reflection and future work):**

  What are the current limitations of the implemented library (e.g., supported assertion forms, expression coverage, formatting constraints, integration boundaries), and which extensions are technically feasible for future versions?

By addressing these questions, this thesis positions its contribution as the design and implementation of a test feedback layer that simplifies and standardises failure reporting for novice programmers, and as a critical reflection on how such a library can be extended and integrated into teaching contexts.

## 1.3. Thesis Structure

This thesis is structured as follows:

- **Chapter 1:** Introduction
- **Chapter 2:** Related Work

- **Chapter 3:** Assertify/Checkify
- **Chapter 4:** Evaluation and Results
- **Chapter 5:** Discussion
- **Chapter 2: Related Work**

  Reviews research on novice programmers' difficulties with compiler and test diagnostics, and summarises prior approaches to improving the structure and usefulness of error feedback. The chapter establishes the conceptual motivation for presenting failures in a more consistent and cognitively simple format.

- **Chapter 3: Analysis of Previous Semester Data**

  Analyses artefacts from earlier course iterations (e.g., submissions, test outcomes, and common failure patterns) to identify recurring sources of errors and to contextualise which kinds of feedback are most frequently encountered by students. This chapter motivates the design goals of a novice-oriented reporting scheme by grounding the problem in a concrete teaching setting.

- **Chapter 4: Problem Analysis**

  Examines typical compiler and testing-framework outputs as they appear in beginner exercises and characterises their technical properties (e.g., verbosity, inconsistency, mixing of essential information with framework internals). Based on this analysis, the chapter derives design requirements for improved test failure reporting, focusing on a stable "tested expression / expected / actual" summary as the primary interface.

- **Chapter 5: Assertify — Design and Implementation**

  Presents the design rationale and implementation of *Assertify* as a lightweight test and assertion library that wraps existing testing infrastructures. It explains how a consistent `expected = actual` output format is produced, including the use of Quotations and Unquote for expression capture and evaluation, as well as wrappers around `UnitTesting` and `FsCheck`. The chapter also discusses implementation choices and their implications for extensibility.

- **Chapter 6: Evaluation and Results**

  Evaluates Assertify primarily from a technical and qualitative perspective, focusing on the properties the implementation achieves (e.g., consistency of output, coverage of supported assertion forms, integration with unit and property-based testing workflows). Where applicable, the chapter contrasts Assertify output with baseline framework output to illustrate differences in interpretability, while clearly separating observations from generalisable claims.

- **Chapter 7: Discussion and Future Work**

Reflects on the contribution and limitations of Assertify and outlines potential extensions. This includes incremental improvements within the wrapper approach (e.g., broader expression coverage, richer formatting layers, IDE integration), as well as a discussion of what would become possible with more extensive redesign efforts, such as implementing a dedicated testing framework from scratch instead of relying on wrappers. The chapter concludes by positioning Assertify as a step toward more pedagogically aligned testing feedback and summarising open directions for future work.

- **Chapter 8: Conclusion**

  Summarises the thesis contributions, revisits the research questions, and highlights the main insights regarding the feasibility and design space of novice-oriented test feedback.

nnnnnnnThis structure emphasizes the design and implementation contribution of the thesis and concludes with a focused discussion of technical possibilities and future extensions, ranging from incremental refinements of the current library to more fundamental redesigns of testing feedback infrastructures.

# 2. Background and Related Work

## 2.1. Challenges for Novice Programmers

## 2.2. Error Messages and Programming Education

## 2.3. Approaches to Improving Error Messages

## 2.4. Case Study: Rust Error Messages?

# 3. Testify: Design and Goals

## 3.1. Design Principles for Pedagoically Useful Error Messages

## 3.2. Using `Microsoft.FSharp.Quotations` and `Swensen.Unquote` for Expression Capture

## 3.3. Assertify: Wrapper for Assertion

## 3.4. Checkify: Wrapper for Property Based Tests

# 4. Evalutation and Results

## 4.1. Evaluation Methodology

## 4.2. Results

## 4.3. Threats to Validity

# 5. Discussion and Conclusion

## 5.1. Key findings

## 5.2. Summary of Contributions

## 5.3. Strengths and Limitations

## 5.4. Future Work

# List of Figures

# List of Listings

1.1. Example Ouput of `Assert.AreEqual` . . . . . 1

# List of Tables

# A. My Code

**TODO**

Put detailed code here.

# B. My Ideas

**TODO**

Ideas??